

Representing Colors

Another common use of numbers is to represent colors. You already know that numbers can be used to represent coordinates on a graph. Computer graphics involves making pictures by plotting blobs of color on the equivalent of electronic graph paper. The blob plotted at each coordinate pair is called a *picture element* or, more commonly, *pixel*.

Computer monitors generate color by mixing red, green, and blue lights using the aptly named *RGB color model*. The colors can be represented by a *color cube*, in which each axis represents a *primary* color, as shown in Figure 1-18. A value of 0 means that a particular light is off, and 1 means that it's as bright as it can get.

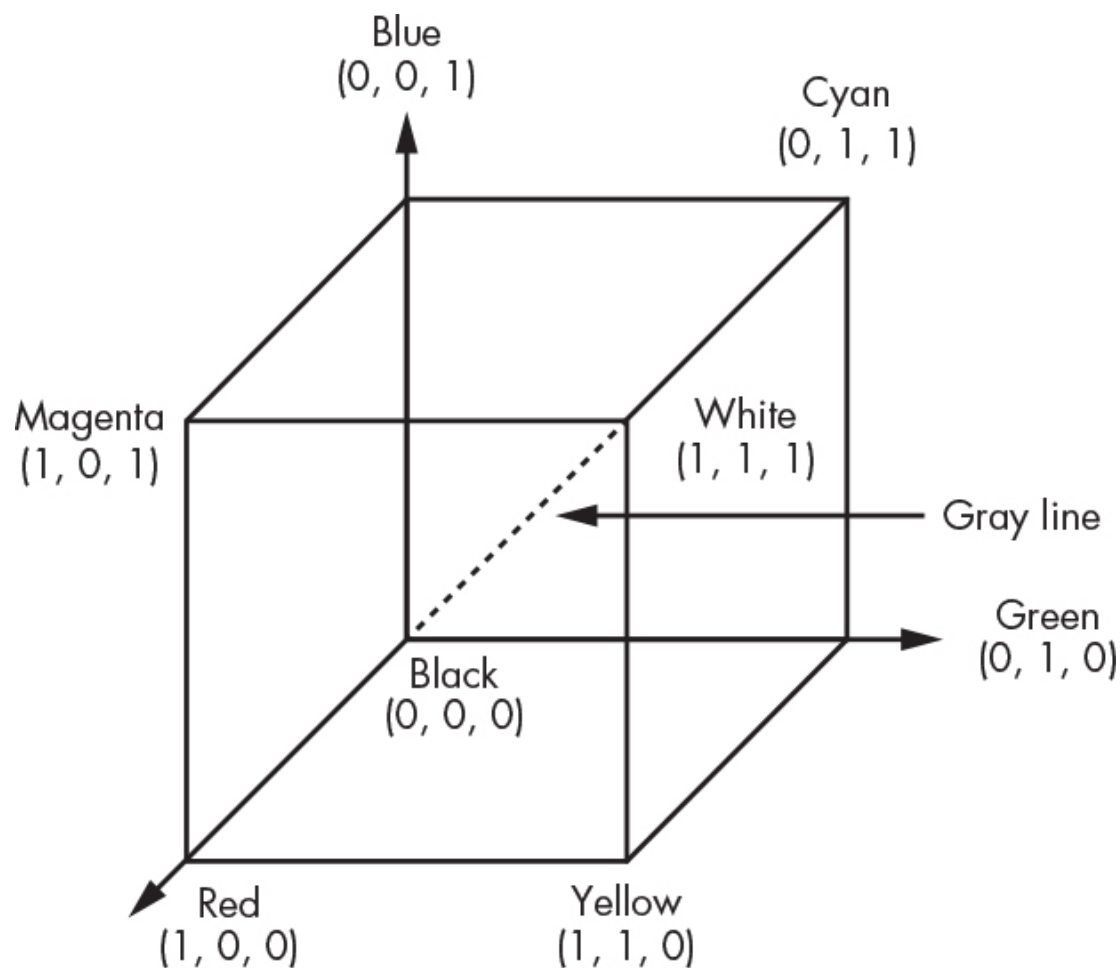


Figure 1-18: An RGB color cube

You can see that the color is black if no lights are on and white if all of the lights are fully up, meaning they're at maximum brightness. A shade of red results

if only the red light is on. Mixing red and green creates yellow. Gray results from setting all three lights to the same level. This way of mixing colors is called an *additive* color system, as adding the primaries produces different colors. Figure 1-19 shows the coordinates of a few colors in the color cube.

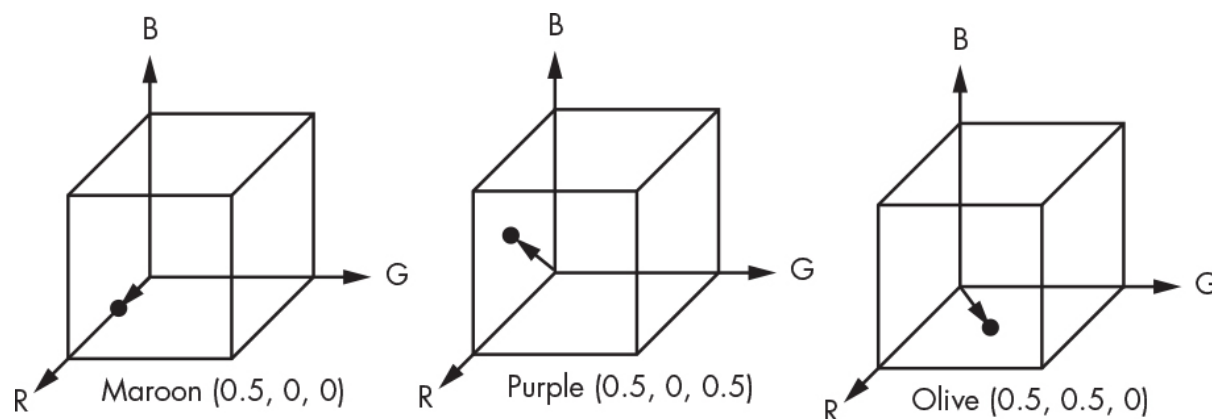


Figure 1-19: RGB color cube examples

If you've tried your hand at painting, you're probably more familiar with a *subtractive* color system, in which the primaries are cyan, magenta, and yellow. A subtractive color system produces colors by removing wavelengths from white light rather than by adding colored light as in the additive system. While neither color system can produce all of the colors that your eye can see, the subtractive system can produce more than the additive system. A whole set of *prepress* technologies exists to make it possible for artists to work on computer monitors and still have their designs look correct when printed in magazines. If you're really interested in color, read *A Field Guide to Digital Color* by Maureen Stone.

The human eye is a very messy piece of machinery that evolved for survival, not computing. It can distinguish around 10 million colors, but it's not linear; doubling the light level doesn't necessarily translate to doubling perceived brightness. Even worse, the eye's response changes slowly over time in response to overall light level. This is called *dark adaptation*. And the response is different for different colors; the eye is very sensitive to changes in green but relatively insensitive to changes in blue, a phenomenon that was exploited in the National Television System Committee (NTSC) standard. Modern computers have settled on rounding up 10 million to the nearest power of 2 and using 24 bits to represent color. These 24 bits are divided up into three 8-bit fields, one for each of the color primaries.

You might have noticed that there is no name for 24 bits in Table 1-10. That's because modern computers aren't designed to operate on 24-bit units (although

there were a few 24-bit machines, such as the Honeywell DDP-224). As a result, colors get packed into the nearest standard size, which is 32 bits (*long word*), as shown in Figure 1-20.

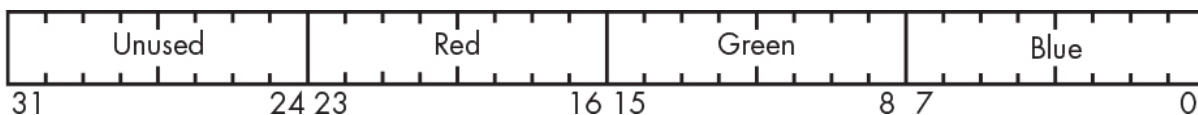


Figure 1-20: RGB color packing

You can see that this scheme leaves 8 unused bits for every color. That’s a lot, considering that computer monitors today have in excess of 8 million pixels. We can’t just let those bits go to waste, so what can we do with them? The answer is that we can use them for something that’s missing from our discussion of color above: *transparency*, meaning how much you can “see through” the color. So far, we’ve only discussed opaque colors, but those can’t be used for rose-colored glasses, for example.

Adding Transparency

In early animated movies, each frame was drawn by hand. Not only was this very labor-intensive, but there was also a lot of visual “jitter,” because it was impossible to exactly reproduce the background on each frame. American animators John Bray (1879–1978) and Earl Hurd (1880–1940) solved this problem with their invention of *cel animation* in 1915. In cel animation, moving characters were drawn on transparent sheets of celluloid, which could then be moved over a static background image.

Although computer animation traces its roots back to the 1940s, it really took off in the 1970s and 1980s. Computers weren’t fast enough back then to do everything that movie directors wanted (and likely never will be because, well, directors). And a mechanism was needed to combine objects generated by different algorithms. Like cel animation, transparency allows for *compositing*, or combining together images from different sources. You’re probably familiar with this concept if you’ve ever played with an image editor like GIMP or Photoshop.

In 1984, Tom Duff and Thomas Porter at Lucasfilm invented a way to implement transparency and compositing that has since become standard. They added a transparency value called *alpha* (α) to each pixel. The α is a mathematical value between 0 and 1, where 0 means that a color is completely transparent and 1 means that a color is completely opaque. A set of *compositing algebra* equations define how colors with different alphas combine to produce new colors.